

Recurrent Neural Networks with PyTorch

A Closed-Set Question–Answering System from Tokens to Confidence-Aware Predictions

Abstract

A recurrent neural network (RNN) processes an ordered sequence while carrying a hidden state from one time step to the next. These notes build a small question–answering application around that principle. The project contains 90 aligned question–answer rows, a 324-token vocabulary, a 50-dimensional trainable embedding, a one-layer RNN with hidden width 64, and a 324-class output layer. Every transformation is made explicit: text normalization, tokenization, vocabulary construction, unknown-token handling, tensor creation, batching, recurrence, cross-entropy training, Adam updates, softmax conversion, and threshold-based rejection. A numerical recurrence is expanded by hand before compact notation is introduced, and every PyTorch tensor shape is traced. The axis error corrected by `batch_first=True` is reproduced precisely. Important limitations are also corrected: an omitted initial state is zero rather than random, the system is a closed-set token classifier rather than a generative answering model, and maximum softmax probability is not a reliable proof that an unseen question is known. Safer answer-class and padded-batch designs provide clear extension paths.

Contents

1	Purpose and complete system contract	2
1.1	The practical task	2
1.2	End-to-end data flow	3
2	Why sequence order requires state	4
2.1	Feedforward and recurrent computation	4
2.2	Unfolding through time	4
3	The recurrent equation, expanded before compact notation	5
3.1	Purpose and dimensions	5
3.2	A complete numerical recurrence	6
4	From text to vocabulary indices	7
4.1	Normalization and tokenization	7
4.2	Unknown tokens	7
5	Embedding lookup: IDs become trainable vectors	8
5.1	Purpose and shape	8
5.2	Connection to one-hot encoding	8
6	Dataset and DataLoader responsibilities	9
6.1	Dataset object state	9

7	The PyTorch RNN architecture	10
7.1	Layer responsibilities	10
7.2	Complete tensor trace	11
7.3	Parameter count	11
8	Debugging sequence and batch axes	12
8.1	The deceptive wrong shape	12
9	Training objective and parameter updates	13
9.1	Cross-entropy from logits to one target	13
9.2	Twenty-epoch optimization loop	13
10	Prediction and confidence-aware rejection	15
10.1	From question to returned token	15
10.2	Why maximum softmax is not knowledge	16
11	Correct output designs for multiword answers	16
11.1	Three different tasks	16
11.2	A corrected answer-class model	17
12	Variable-length batches and padding	17
12.1	Why padding is needed	17
13	Failure modes and disciplined repairs	18
13.1	Further engineering improvements	19
14	Review questions and exercises	19

1 Purpose and complete system contract

1.1 The practical task

The application receives an English question and returns either one answer from a small learned answer set or the string `I don't know`. Training data are stored in two aligned columns: one question and one answer per row. Representative pairs include the following.

Question	Stored answer token
What is the capital of France?	Paris
What is the capital of Germany?	Berlin
Who wrote To Kill a Mockingbird?	Harper-Lee
What is the largest planet in our solar system?	Jupiter
What is the boiling point of water in Celsius?	100
Who painted the Mona Lisa?	Leonardo-da-Vinci
What is the square root of 64?	8
What is the chemical symbol for gold?	Au
Which year did World War II end?	1945
What is the longest river in the world?	Nile
Who developed the theory of relativity?	Albert-Einstein
Which planet is known as the Red Planet?	Mars

Hyphens are deliberate. The simple tokenizer splits only at spaces, so `albert-einstein` remains one token and can be represented by one output class. Without that convention, a one-step classifier could not emit a two-word answer.

Definition

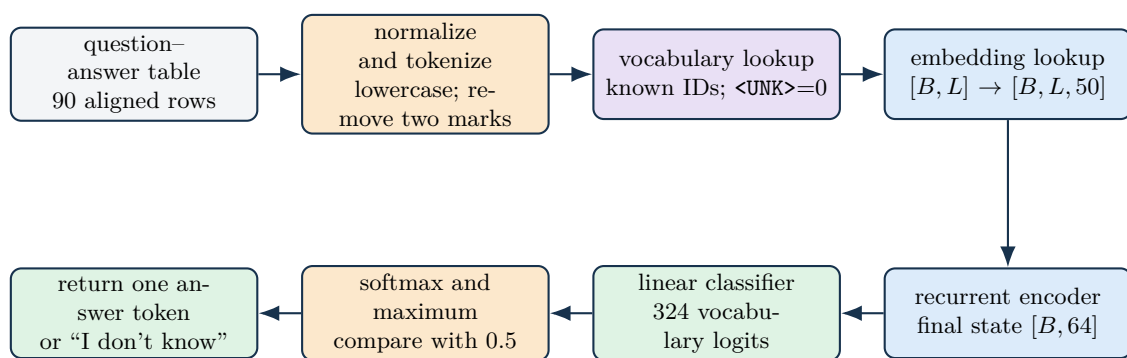
A **closed-set classifier** chooses one label from a fixed set known during training. An **open-ended generator** can produce a sequence of new tokens. The present architecture is a closed-set classifier, even though its input and labels are words.

1.2 End-to-end data flow

The complete path is

question text $\rightarrow$ tokens $\rightarrow$ integer IDs $\rightarrow$ embeddings $\rightarrow$ RNN state $\rightarrow$ logits $\rightarrow$ decision.

A **token** is one unit produced by tokenization. An **integer ID** is an arbitrary vocabulary index. An **embedding** is a learned dense vector. A **hidden state** is the RNN's numerical summary of the sequence prefix. A **logit** is an unrestricted real-valued class score. The final decision either maps the highest-scoring class back to a token or rejects it when a confidence rule fails.



This is closed-set answer classification, not open-ended generation. Hyphenated answer strings such as `albert-einstein` remain one token so one output class can represent them.

Figure 1: The complete application contract connects tabular data, text processing, tensor conversion, recurrence, classification, and a reject option. [Technical reference](#).

Five implementation stages keep responsibilities separate:

1. load and inspect the aligned question-answer table;
2. normalize text, build a vocabulary, and map words to IDs;
3. define the embedding, recurrent layer, and output layer;
4. optimize parameters on the training pairs;
5. convert logits to a returned answer or a rejection.

Takeaway

The model does not search a knowledge base at prediction time. It learns a function from a question sequence to one class in a fixed output set.

2 Why sequence order requires state

2.1 Feedforward and recurrent computation

A conventional feedforward network treats one input vector as a complete, independent example. A sequence instead contains ordered elements. In text, changing word order can change meaning, and the interpretation of a later word may depend on earlier words.

Definition

Sequential data contain observations whose order carries information. Examples include words in a sentence, audio samples, sensor readings, and values in a time series. A **time step** is one position in such an ordered sequence.

An RNN carries a state h_{t-1} from the previous time step into the current computation. This state supplies context unavailable to an independent feedforward transformation.

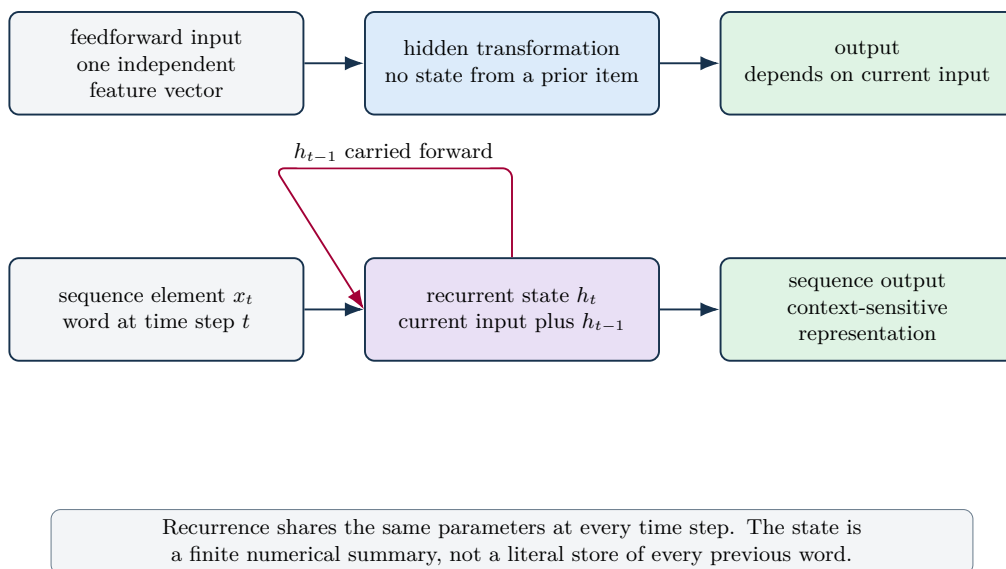


Figure 2: A recurrent cell combines the current sequence element with a carried state; its parameters are reused at every time step. [Technical reference](#).

The state is often described informally as memory. More precisely, it is a finite-dimensional vector trained to retain information useful to the objective. It can forget details, distort long-range information, and cannot be assumed to contain a human-readable record.

2.2 Unfolding through time

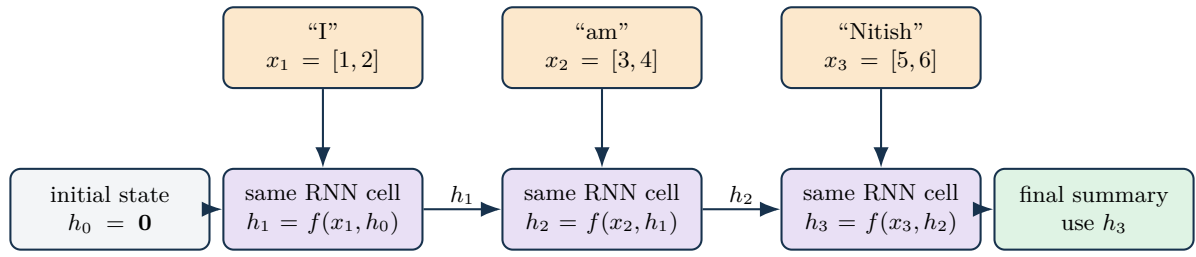
Consider the three-token sentence “I am Nitish.” For intuition only, assign two-dimensional vectors

$$x_1 = [1, 2], \quad x_2 = [3, 4], \quad x_3 = [5, 6].$$

Here x_t denotes the vector at time step t . The values are hypothetical; real embeddings are learned and have dimension 50 in this project.

The recurrence processes the sequence in order:

$$\begin{aligned} h_1 &= f(x_1, h_0), \\ h_2 &= f(x_2, h_1), \\ h_3 &= f(x_3, h_2). \end{aligned}$$



Unfolding draws one conceptual copy per time step, but all copies use the same input, recurrent, and bias parameters. PyTorch uses a zero initial state when `h_0` is omitted.

Figure 3: Unfolding makes the temporal dependency visible while preserving one shared parameter set across all three positions. [Technical reference](#).

The function f is the same RNN cell at every line. The symbols h_0, h_1, h_2, h_3 denote successive hidden states. The final state h_3 summarizes the complete three-token prefix and can feed a classifier.

Technical precision

When `torch.nn.RNN` receives no explicit `h_0`, PyTorch initializes it to zeros. A deterministic execution check confirmed that an omitted state and an explicit zero state produced a maximum absolute difference of exactly 0. A random initial state is neither required nor the default.

3 The recurrent equation, expanded before compact notation

3.1 Purpose and dimensions

For one `tanh` RNN layer, PyTorch computes

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh}).$$

Every term has a specific responsibility:

- $x_t \in \mathbb{R}^{B \times E}$ is the current embedded token for B sequences and embedding width E ;
- $W_{ih} \in \mathbb{R}^{H \times E}$ maps input features to hidden width H ;
- $h_{t-1} \in \mathbb{R}^{B \times H}$ is the preceding hidden state;
- $W_{hh} \in \mathbb{R}^{H \times H}$ maps the old state back into hidden space;
- $b_{ih}, b_{hh} \in \mathbb{R}^H$ are learned bias vectors;
- $h_t \in \mathbb{R}^{B \times H}$ is the updated state;
- $\tanh$ is applied element by element and bounds every state component between -1 and 1 .

The equation computes a new state from current evidence and remembered context. Its importance is parameter sharing: W_{ih} , W_{hh} , b_{ih} , and b_{hh} do not change with t .

3.2 A complete numerical recurrence

Use $B = 1$, $E = 2$, $H = 2$, and

$$W_{ih} = \begin{bmatrix} 0.50 & -0.25 \\ 0.10 & 0.40 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.30 & 0.20 \\ -0.20 & 0.25 \end{bmatrix},$$

$$b_{ih} = \begin{bmatrix} 0.05 & -0.10 \end{bmatrix}, \quad b_{hh} = \begin{bmatrix} 0.01 & 0.02 \end{bmatrix}, \quad h_0 = \begin{bmatrix} 0 & 0 \end{bmatrix}.$$

At the first time step, $x_1 = [1, 2]$. Expand each contribution in the same order as the compact formula:

$$x_1 W_{ih}^T = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 0.50 & 0.10 \\ -0.25 & 0.40 \end{bmatrix} = \begin{bmatrix} 0 & 0.90 \end{bmatrix},$$

$$h_0 W_{hh}^T = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad b_{ih} + b_{hh} = \begin{bmatrix} 0.06 & -0.08 \end{bmatrix}.$$

Therefore

$$h_1 = \tanh\left(\begin{bmatrix} 0.06 & 0.82 \end{bmatrix}\right) \approx \begin{bmatrix} 0.059928 & 0.675070 \end{bmatrix}.$$

The first component is close to zero because its weighted input canceled; the second is positive because its pre-activation was 0.82.

Repeating the same operation for $x_2 = [0, 1]$ and $x_3 = [2, -1]$ gives

time step	$h_{t,1}$	$h_{t,2}$
1	0.059928	0.675070
2	-0.036991	0.443663
3	0.882650	-0.160292

The two columns are the two hidden dimensions. A direct `torch.nn.RNN` computation reproduced all six values; the maximum absolute difference from the manual calculation was 7.45058×10^{-9} . This verifies that the expanded arithmetic and compact recurrence are the same operation up to floating-point rounding.

```
import torch
from torch import nn

rnn = nn.RNN(2, 2, batch_first=True)
with torch.no_grad():
    rnn.weight_ih_l0.copy_(
        torch.tensor([[0.50, -0.25], [0.10, 0.40]])
    )
    rnn.weight_hh_l0.copy_(
        torch.tensor([[0.30, 0.20], [-0.20, 0.25]])
    )
    rnn.bias_ih_l0.copy_(torch.tensor([0.05, -0.10]))
    rnn.bias_hh_l0.copy_(torch.tensor([0.01, 0.02]))

x = torch.tensor([[[1.0, 2.0], [0.0, 1.0], [2.0, -1.0]]])
all_states, final_state = rnn(x)
assert all_states.shape == (1, 3, 2)
assert final_state.shape == (1, 1, 2)
assert torch.allclose(all_states[:, -1], final_state[-1])
```

Listing 1: Verify a manually expanded tanh recurrence against PyTorch

Pitfall

Repeated multiplication through many time steps can make gradients shrink or grow. These are the vanishing- and exploding-gradient problems. Gradient clipping limits explosion; gated architectures such as long short-term memory (LSTM) and gated recurrent units (GRUs) improve long-range state control.

4 From text to vocabulary indices

4.1 Normalization and tokenization

The project tokenizer performs exactly three transformations: lowercase the string, remove question marks, remove apostrophes, and split on whitespace. Its purpose is to make superficially different forms such as `France` and `france?` share one vocabulary entry.

```
def tokenize(text):
    text = text.lower()
    text = text.replace("?", "")
    text = text.replace("'", "")
    return text.split()

vocab = {"<UNK>": 0}

def build_vocab(row):
    question_tokens = tokenize(row["question"])
    answer_tokens = tokenize(row["answer"])
    for token in question_tokens + answer_tokens:
        if token not in vocab:
            vocab[token] = len(vocab)

# Apply once to every aligned question--answer row.
df.apply(build_vocab, axis=1)
assert len(vocab) == 324
```

Listing 2: Normalize text and build an insertion-ordered vocabulary

A **vocabulary** is the mapping from token strings to integer IDs. The first entries are

`<UNK>` $\mapsto$ 0, `what` $\mapsto$ 1, `is` $\mapsto$ 2, `the` $\mapsto$ 3, `capital` $\mapsto$ 4, `of` $\mapsto$ 5.

The values are identifiers, not magnitudes. ID 5 is not semantically “larger” than ID 2.

4.2 Unknown tokens

Prediction questions can contain words absent from training. Every such word maps to the special `<UNK>` ID 0.

```
def text_to_indices(text, vocab):
    indexed_text = []
    for token in tokenize(text):
        if token in vocab:
            indexed_text.append(vocab[token])
        else:
            indexed_text.append(vocab["<UNK>"])
    return indexed_text

assert text_to_indices("What is CampusX?", vocab) == [1, 2, 0]
```

Listing 3: Convert a token sequence to vocabulary indices with an unknown fallback

Pitfall

The tokenizer is intentionally small, not linguistically complete. It leaves commas, periods, slashes, Unicode quotation marks, and many other forms untouched. It also maps every unseen word to the same vector, so different unknown words become indistinguishable.

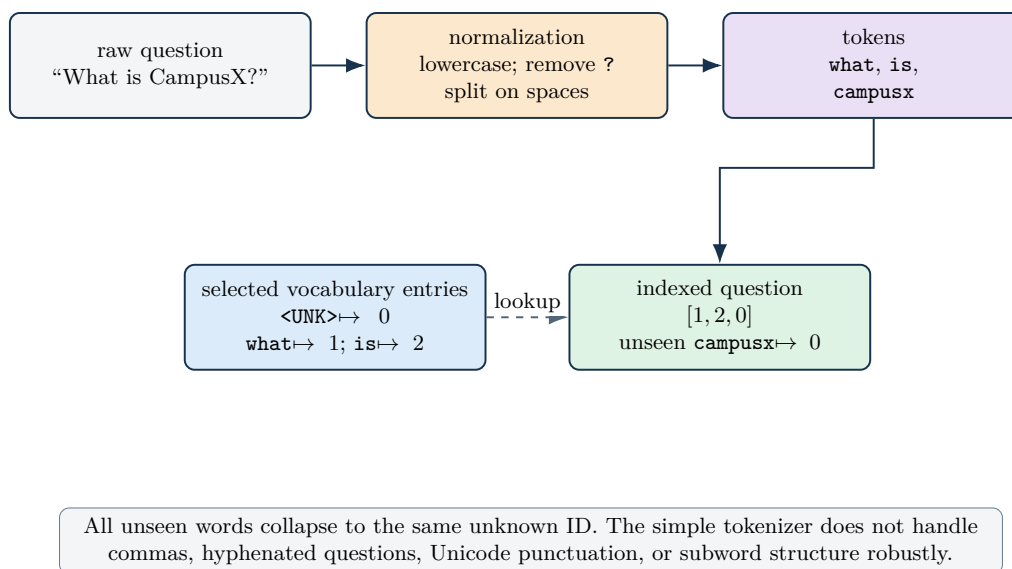


Figure 4: Lowercasing and vocabulary lookup turn the unseen word `campusx` into the common unknown ID 0. [Technical reference](#).

5 Embedding lookup: IDs become trainable vectors

5.1 Purpose and shape

An RNN cannot learn useful language patterns directly from arbitrary integer IDs. An embedding table assigns one trainable vector to each token:

$$E \in \mathbb{R}^{V \times D}.$$

Here E is the embedding matrix, $V = 324$ is vocabulary size, and $D = 50$ is embedding width. For an ID tensor

$$I \in \mathbb{N}^{B \times L},$$

where B is batch size and L is sequence length, `nn.Embedding` returns

$$X = E[I] \in \mathbb{R}^{B \times L \times 50}.$$

The expression $E[I]$ means row lookup. For one token ID i , the returned vector is row $E_{i,:}$.

5.2 Connection to one-hot encoding

Let $e_i \in \{0, 1\}^V$ contain a single 1 at position i . Then

$$e_i^\top E = E_{i,:}.$$

This equation computes the same vector as an embedding lookup. It matters because lookup avoids constructing a mostly zero V -dimensional vector.

For example, if ID 2 selects

$$E_{2,:} = \begin{bmatrix} 0.7 & 0.3 & -0.4 \end{bmatrix},$$

then both `embedding(torch.tensor(2))` and one-hot matrix multiplication return the same three numbers. A deterministic execution check confirmed exact tensor equality for this example.

Shape and state check

A six-word unbatched question produces shape `[6, 50]`. After a `DataLoader` adds batch size one, the same example has shape `[1, 6]` before embedding and `[1, 6, 50]` after embedding. Keep the two situations distinct.

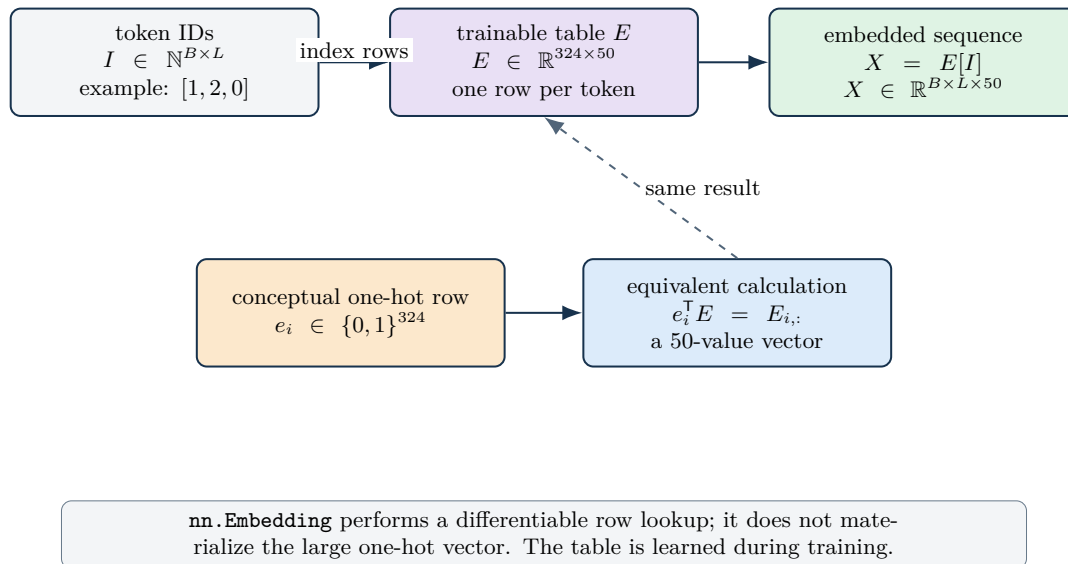


Figure 5: Embedding lookup changes the representation from discrete IDs to dense, learned vectors without changing batch or sequence axes. [Technical reference](#).

6 Dataset and DataLoader responsibilities

6.1 Dataset object state

A `Dataset` owns the data frame and vocabulary. Its invariants are:

- every row contains an aligned question and answer;
- `__len__` returns the row count, 90;
- `__getitem__(i)` returns integer tensors for exactly row i ;
- answer strings remain one token because multiword labels use hyphens.

```
import torch
from torch.utils.data import DataLoader, Dataset

class QADataset(Dataset):
    def __init__(self, frame, vocab):
        self.frame = frame
        self.vocab = vocab

    def __len__(self):
        return self.frame.shape[0]

    def __getitem__(self, index):
        question = text_to_indices(
            self.frame.iloc[index]["question"], self.vocab
        )
        answer = text_to_indices(
            self.frame.iloc[index]["answer"], self.vocab
        )
        return torch.tensor(question), torch.tensor(answer)

qa_dataset = QADataset(df, vocab)
qa_loader = DataLoader(
    qa_dataset, batch_size=1, shuffle=True
)
```

Listing 4: Represent one aligned question–answer row as integer tensors

For the first row, the intended tensors are

$$q_0 = [1, 2, 3, 4, 5, 6], \quad y_0 = [7],$$

corresponding to “what is the capital of france” and “paris.” The DataLoader adds a leading batch axis:

$$Q \in \mathbb{N}^{1 \times L}, \quad Y \in \mathbb{N}^{1 \times 1}.$$

The symbol L changes from question to question. The label tensor uses integer type because it stores a class index.

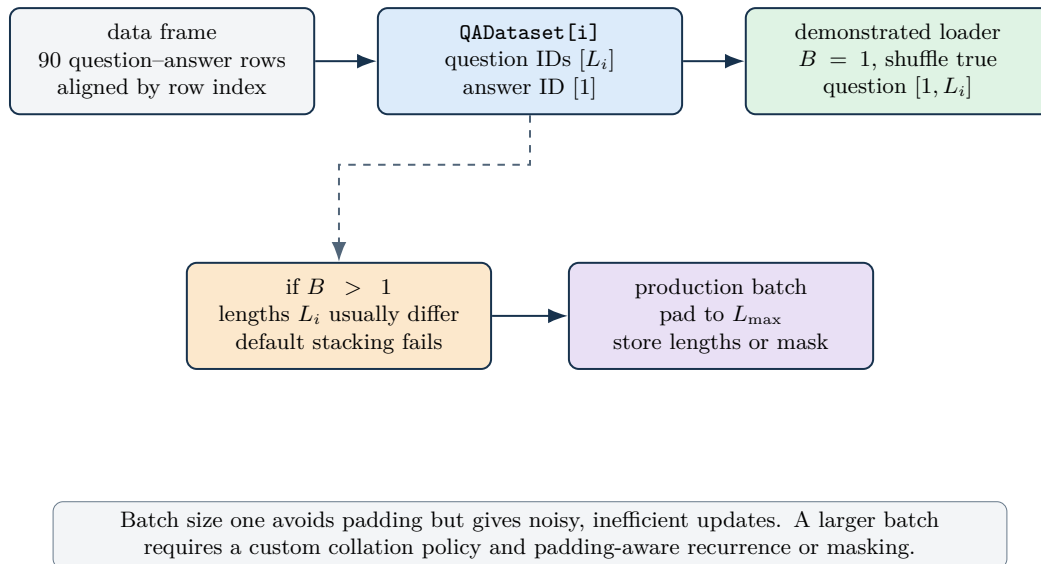


Figure 6: Batch size one avoids variable-length stacking, while larger batches require padding and an explicit length or mask contract. [Technical reference](#).

Pitfall

Default collation cannot stack a length-2 tensor with a length-3 tensor. A real execution produced: **stack expects each tensor to be equal size**. Batch size one hides this issue; it does not solve variable-length batching.

7 The PyTorch RNN architecture

7.1 Layer responsibilities

The architecture has three trainable components:

Embedding. Maps each vocabulary ID to 50 learned values.

RNN. Processes 50 values per time step and maintains a 64-value hidden state.

Linear classifier. Converts the final 64-value state to 324 logits, one for every vocabulary token.

```

import torch.nn as nn

class SimpleRNN(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, 50)
        self.rnn = nn.RNN(
            input_size=50,
            hidden_size=64,
            batch_first=True,
        )
        self.fc = nn.Linear(64, vocab_size)

    def forward(self, question):
        embedded = self.embedding(question)
        all_states, h_n = self.rnn(embedded)
        final_state = h_n[-1]
        logits = self.fc(final_state)
        return logits

```

Listing 5: Define the one-layer recurrent token classifier with an explicit batch-first contract

The modules are not placed in `nn.Sequential`. An RNN returns two objects—all time-step outputs and the final hidden-state stack—whereas a plain sequential container passes one object from each module to the next. The custom `forward` method explicitly selects the final state.

7.2 Complete tensor trace

For $B = 1$, $L = 6$, $V = 324$, $D = 50$, and $H = 64$:

$$\begin{aligned}
 Q &: [1, 6], \\
 \text{Embedding}(Q) &: [1, 6, 50], \\
 \text{all_states} &: [1, 6, 64], \\
 \text{h_n} &: [1, 1, 64], \\
 \text{h_n}[-1] &: [1, 64], \\
 \text{Linear}(\text{h_n}[-1]) &: [1, 324].
 \end{aligned}$$

The first axis of `h_n` indexes recurrent layers and directions, not time. For a one-layer, one-direction RNN it has size one. Indexing with `h_n[-1]` preserves the batch axis even when $B = 1$.

7.3 Parameter count

The embedding owns

$$P_{\text{emb}} = VD = 324 \cdot 50 = 16,200$$

parameters. The product counts one 50-value row per vocabulary token.

A one-layer RNN owns an input matrix, recurrent matrix, and two bias vectors:

$$P_{\text{rnn}} = HD + H^2 + 2H = 64 \cdot 50 + 64^2 + 2 \cdot 64 = 7,424.$$

The first product maps embeddings; the square matrix maps the old hidden state; the final term counts both PyTorch bias vectors.

The classifier owns

$$P_{\text{fc}} = HV + V = 64 \cdot 324 + 324 = 21,060.$$

The product counts weights and the final V counts biases. Therefore

$$P_{\text{total}} = 16,200 + 7,424 + 21,060 = 44,684.$$

A direct parameter count matched all four values. This matters because even a small demonstration has tens of thousands of adjustable values for only 90 rows, making memorization easy.

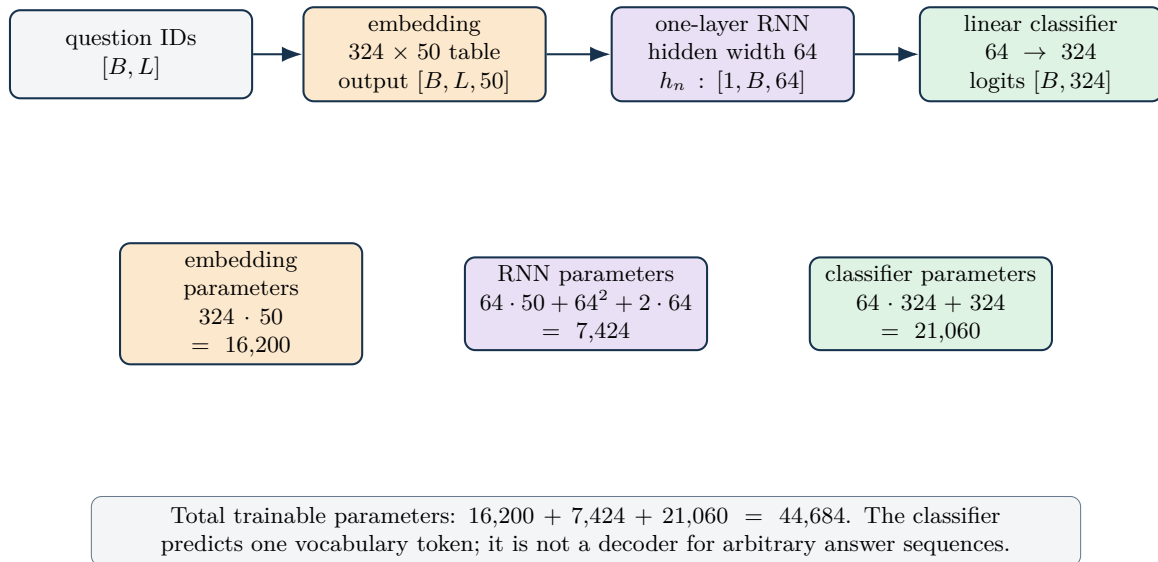


Figure 7: The project maps question IDs to a 64-value final state and then to 324 vocabulary logits. [Technical reference.](#)

8 Debugging sequence and batch axes

8.1 The deceptive wrong shape

Without `batch_first=True`, `nn.RNN` expects $[L, B, D]$. Feeding an embedded tensor shaped $[1, 6, 50]$ then means

$$L = 1, \quad B = 6,$$

not one six-token question. The output of all time steps still has visible shape $[1, 6, 64]$, so checking only that tensor can miss the bug. The final hidden stack exposes the error:

$$\text{wrong } h_n : [1, 6, 64] \longrightarrow \text{wrong logits } [1, 6, 324].$$

With the correct option,

$$\text{correct } h_n : [1, 1, 64] \longrightarrow \text{correct logits } [1, 324].$$

The first failed training attempt exposed this axis error through cross-entropy. For a nine-token question, the model returned $[1, 9, 324]$. Cross-entropy interpreted 9 as the class count and 324 as an extra spatial dimension, then reported **Expected target size $[1, 324]$, got $[1, 1]$** . Indexing the answer changed only the final part to **got $[1]$** ; it could not repair the malformed logits. The model axis contract had to be corrected before adjusting the label.

```
question = torch.tensor([[1, 2, 3, 4, 5, 6]])
embedded = model.embedding(question)
all_states, h_n = model.rnn(embedded)
final_state = h_n[-1]
logits = model.fc(final_state)

assert question.shape == (1, 6)
assert embedded.shape == (1, 6, 50)
assert all_states.shape == (1, 6, 64)
assert h_n.shape == (1, 1, 64)
assert final_state.shape == (1, 64)
assert logits.shape == (1, 324)
assert torch.allclose(all_states[:, -1], final_state)
```

Listing 6: Assert every axis at the corrected recurrent boundary

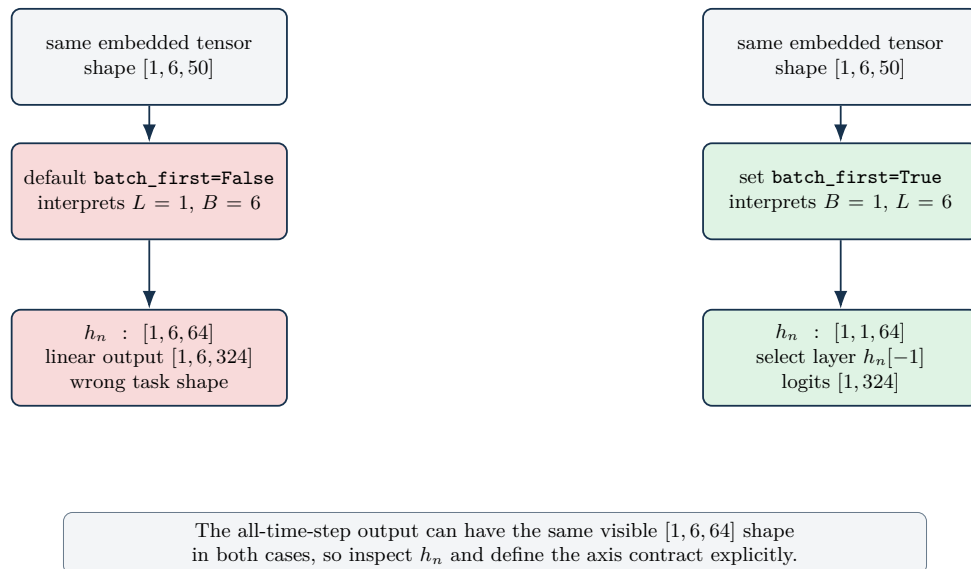


Figure 8: The same three numbers can describe opposite axis meanings; inspecting `h_n` reveals whether six is batch size or sequence length. [Technical reference](#).

Pitfall

Calling `h_n.squeeze()` removes every size-one axis. At $B = 1$, shape $[1, 1, 64]$ becomes $[64]$ and silently loses the batch axis. Use `h_n[-1]` or, only for a known one-layer layout, `h_n.squeeze(0)`.

9 Training objective and parameter updates

9.1 Cross-entropy from logits to one target

For one question, let $z \in \mathbb{R}^V$ be its 324 logits and let $y \in \{0, \dots, V-1\}$ be the correct answer ID. Softmax defines

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^V e^{z_k}}, \quad j \in \{1, \dots, V\}.$$

The symbol p_j is the normalized probability assigned to class j . Every p_j is nonnegative and the probabilities sum to one. Cross-entropy for the correct class is

$$\ell(z, y) = -\log p_y.$$

The loss is small when the correct class receives high probability and large when it receives low probability. `nn.CrossEntropyLoss` accepts raw logits and applies the numerically stable log-softmax calculation internally; do not apply softmax first during training.

The required shape and type contract is

$$z : [B, V], \quad y : [B], \quad y \text{ has integer type.}$$

A target with shape $[B, 2]$ is a multi-target tensor and does not match this classifier. A floating-point class target also fails.

9.2 Twenty-epoch optimization loop

The project uses batch size one, cross-entropy, Adam, and 20 epochs on CPU. A learning rate of 0.001 gives the optimizer's update scale.

```

model = SimpleRNN(len(vocab))
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
epochs = 20

for epoch in range(epochs):
    model.train()
    total_loss = 0.0

    for questions, answers in qa_loader:
        optimizer.zero_grad(set_to_none=True)
        logits = model(questions)

        # [B, 1] -> [B]; every stored answer is one token.
        targets = answers.squeeze(1)
        loss = criterion(logits, targets)

        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch {epoch + 1}: total loss = {total_loss:.4f}")

```

Listing 7: Train the recurrent classifier while preserving the label shape contract

The inner loop performs six operations in order: clear stale gradients, compute logits, compute loss, run backpropagation through time, update parameters, and accumulate a reporting value. The printed quantity is a sum across 90 one-example batches. Dividing by 90 would produce mean loss per row.

The final six printed totals were:

epoch	15	16	17	18	19	20
total loss	25.718873	21.447655	17.891827	15.389441	13.326826	11.588831

Each number is the sum of 90 batch losses for that epoch. Their decline shows increasingly confident fitting of the training pairs; it is not a validation metric.

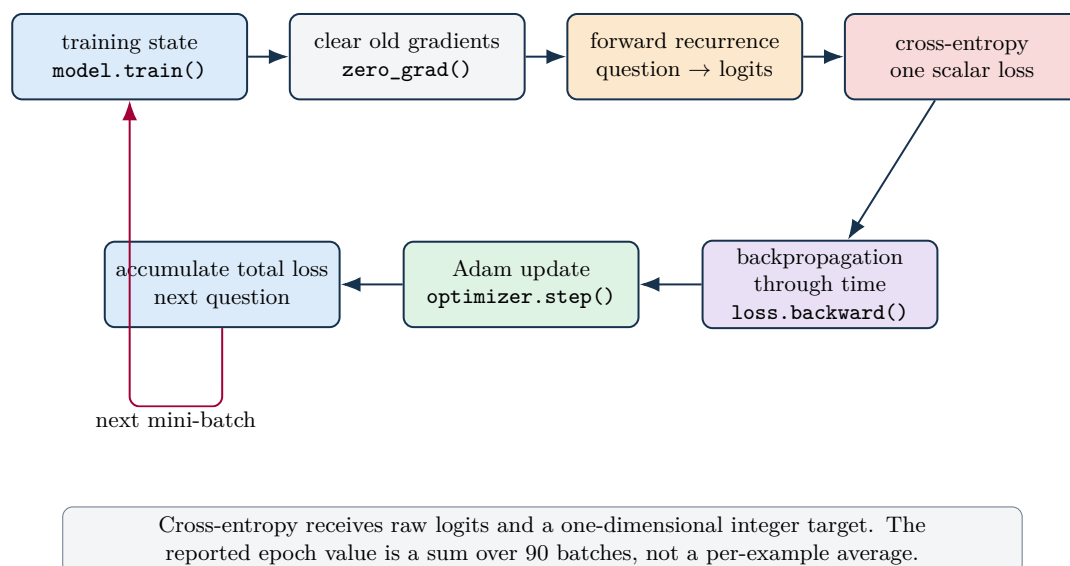


Figure 9: Training alternates forward recurrence, cross-entropy, backpropagation through time, and an Adam update for every mini-batch. [Technical reference](#).

A deterministic source-shape smoke test executed a real backward pass and Adam update. It produced loss 6.267275 before the update, classifier gradient norm 3.615216, and a maximum classifier-weight

change of approximately 0.001000002. These values validate mechanics on synthetic token IDs; they are not an accuracy claim for the 90-row application.

Technical precision

A decreasing total training loss proves that optimization is fitting the training rows. It does not establish performance on unseen questions. A train/validation/test split or cross-validation is necessary for a generalization estimate.

10 Prediction and confidence-aware rejection

10.1 From question to returned token

Prediction repeats training preprocessing, adds a batch axis, computes logits, converts them to probabilities, and selects the largest value. A threshold of 0.5 controls a simple reject option.

```
id_to_token = {index: token for token, index in vocab.items()}

def predict_with_confidence(model, question_text, threshold=0.5):
    question_ids = text_to_indices(question_text, vocab)
    question = torch.tensor(question_ids).unsqueeze(0)

    model.eval()
    with torch.inference_mode():
        logits = model(question)
        probabilities = torch.softmax(logits, dim=1)
        confidence, predicted_id = probabilities.max(dim=1)

    if confidence.item() < threshold:
        return "I don't know"
    return id_to_token[int(predicted_id.item())]
```

Listing 8: Run state-safe prediction with an explicit vocabulary inverse and reject option

For “What is CampusX?”, the token path is

$$[\text{what, is, campusx}] \longrightarrow [1, 2, 0] \longrightarrow [1, 3, 50] \longrightarrow [1, 324].$$

The final tensor contains logits, not probabilities. Softmax preserves shape $[1, 324]$. The maximum operation returns both the largest probability and its class index.

The project behavior includes:

Question	Returned answer
What is the capital of France?	paris
Who developed the theory of relativity?	albert-einstein
What is the capital city of the European country France?	paris
What is the largest planet in our solar system?	jupiter
What is CampusX?	I don't know

The paraphrase result illustrates some tolerance to changed word sequence and unknown words. It is one observed behavior on a tiny memorization-oriented dataset, not evidence of broad language understanding.

For “What is CampusX?”, the largest raw softmax value was 0.0367 at vocabulary index 16. That index mapped to **harper-lee**; the classifier still had to choose a class internally. Because $0.0367 < 0.5$, the reject rule suppressed that arbitrary token and returned **I don't know**. This intermediate result explains both the usefulness and the fragility of the threshold.

10.2 Why maximum softmax is not knowledge

A neural classifier must assign its probability mass somewhere. It can be highly confident on inputs unlike its training distribution. In a controlled six-pair answer-class smoke test, “What is CampusX?” received the wrong answer **H2O** with maximum softmax probability 0.853794—well above a 0.5 threshold. The same model correctly returned **Paris** for a known question with probability 0.996921. Thus thresholding can work on selected examples but cannot by itself certify that the input is in-domain.

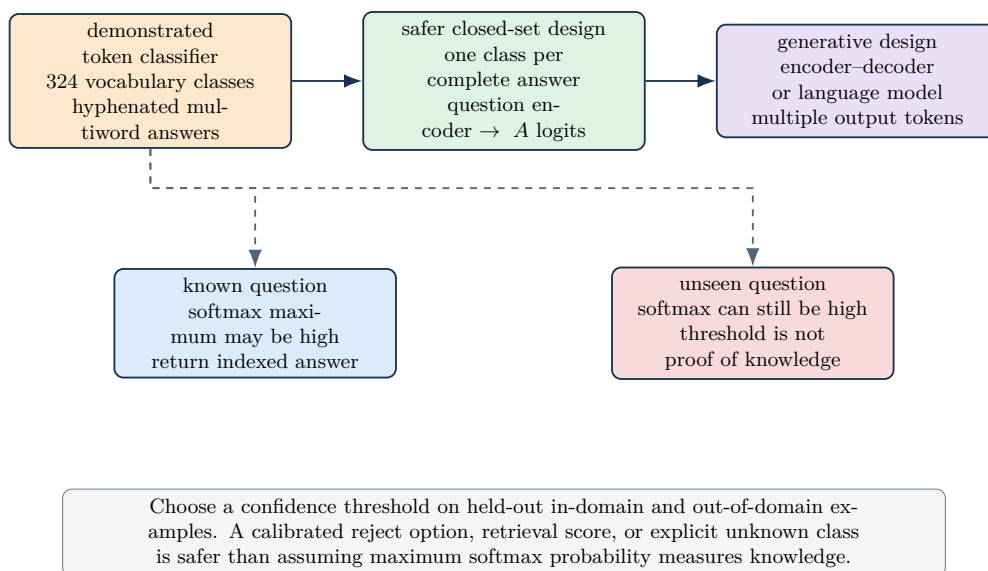


Figure 10: A token classifier, a complete-answer classifier, and a generative decoder solve different tasks; all require a validated uncertainty policy. [Technical reference](#).

A safer reject policy should be selected on two held-out sets:

- in-domain questions with known answers, including paraphrases;
- out-of-domain questions that should be rejected.

Measure both answer accuracy and rejection errors. Calibration methods, retrieval similarity, an explicit unknown class, or conformal prediction can strengthen the policy.

11 Correct output designs for multiword answers

11.1 Three different tasks

The output layer determines what the system can express.

Vocabulary-token classification. Output width is $V = 324$. One prediction is one token. Hyphenation compresses a phrase into one artificial token.

Complete-answer classification. Every distinct answer string receives one class ID. Output width is A , the number of unique answers. Spaces are preserved in the returned string.

Sequence generation. A decoder emits several tokens, usually until an end token. This is required for novel or variable-length answers.

For the present closed set, complete-answer classification is cleaner than mixing question words and answer labels in one 324-class vocabulary. Question-only words can never be correct answers yet occupy output neurons in the token design.

11.2 A corrected answer-class model

Let `answer_to_id` map each entire answer string to one class. The model then predicts A answer logits instead of V token logits. Padding-aware recurrence makes larger batches possible.

```
from torch.nn.utils.rnn import pack_padded_sequence

class AnswerClassRNN(nn.Module):
    def __init__(self, vocab_size, answer_count):
        super().__init__()
        self.embedding = nn.Embedding(
            vocab_size, 50, padding_idx=0
        )
        self.rnn = nn.RNN(50, 64, batch_first=True)
        self.fc = nn.Linear(64, answer_count)

    def forward(self, token_ids, lengths):
        embedded = self.embedding(token_ids)
        packed = pack_padded_sequence(
            embedded,
            lengths.cpu(),
            batch_first=True,
            enforce_sorted=False,
        )
        _, h_n = self.rnn(packed)
        return self.fc(h_n[-1])
```

Listing 9: Encode variable-length questions and classify complete answer strings

A six-pair deterministic smoke test used this design with padded question shape $[6, 9]$ and six answer classes. It reached 100% training accuracy and loss 0.003729 after 14 epochs. This proves the code can memorize six rows; it does not estimate validation accuracy. The distinction between successful execution and generalization must remain explicit.

12 Variable-length batches and padding

12.1 Why padding is needed

Suppose two questions contain lengths $L_1 = 6$ and $L_2 = 9$. A dense batch must be rectangular, so shorter sequences are extended to

$$L_{\max} = \max(L_1, L_2) = 9.$$

A special padding ID fills the extra positions. The true lengths remain available so the RNN does not treat padding as meaningful content.

```

from torch.nn.utils.rnn import pad_sequence

PAD_ID = 0

def collate_questions(batch):
    questions, answer_ids = zip(*batch)
    lengths = torch.tensor(
        [len(question) for question in questions]
    )
    padded = pad_sequence(
        questions,
        batch_first=True,
        padding_value=PAD_ID,
    )
    targets = torch.tensor(answer_ids, dtype=torch.long)
    return padded, lengths, targets

loader = DataLoader(
    dataset,
    batch_size=16,
    shuffle=True,
    collate_fn=collate_questions,
)

```

Listing 10: Collate variable-length questions into a padded batch and length vector

The padded tensor has shape $[B, L_{\max}]$, lengths have shape $[B]$, and answer targets have shape $[B]$. `padding_idx=0` keeps the padding embedding fixed, while `padding` prevents padding positions from contributing recurrent steps.

Pitfall

If ID 0 already means `<UNK>`, it cannot simultaneously be a semantically distinct padding token. Use separate IDs such as `<PAD>=0` and `<UNK>=1`, then rebuild the vocabulary consistently.

13 Failure modes and disciplined repairs

Failure	Observable symptom	Structural repair
Wrong axis convention	logits $[1, L, V]$ instead of $[B, V]$	set <code>batch_first=True</code> ; assert <code>h_n</code>
Unrestricted <code>squeeze()</code>	batch axis disappears when $B = 1$	select <code>h_n[-1]</code>
Multi-token target	cross-entropy reports multi-target shape error	one answer class or a decoder
Floating class target	cross-entropy rejects target type	use <code>torch.long</code> class IDs
Out-of-range ID	embedding raises an index error	map unseen words to <code><UNK></code>
Variable lengths with default collation	tensor stacking fails	pad and preserve lengths
Softmax before training loss	objective becomes incorrectly transformed	give raw logits to cross-entropy
No evaluation state	future dropout or normalization behaves incorrectly	call <code>eval()</code> and disable gradients
Threshold chosen on training examples	confident wrong answers pass	tune on held-out known and unknown inputs
Training loss treated as accuracy	memorization is mistaken for generalization	report validation and test metrics

Four failure messages were reproduced directly: multi-target cross-entropy, floating targets, an out-of-range embedding index, and variable-length default collation. Shape assertions should therefore be treated as executable specifications, not optional comments.

13.1 Further engineering improvements

1. Reserve distinct padding and unknown IDs.
2. Normalize punctuation with a tested tokenizer or subword model.
3. Split data by semantic intent so near-duplicate questions do not leak across sets.
4. Use gradient clipping, for example `clip_grad_norm_`, when long sequences cause unstable gradients.
5. Compare a plain RNN with GRU and LSTM baselines.
6. Save the vocabulary, answer mapping, preprocessing configuration, and model state together; a model without its mappings is unusable.
7. Record random seeds and package versions for reproducible experiments.
8. Evaluate paraphrase accuracy, answer accuracy, rejection precision, rejection recall, and calibration.

Takeaway

Most defects in a small sequence system are contract defects: wrong axis meaning, inconsistent token IDs, invalid label shape, ambiguous output semantics, or an unvalidated confidence rule. Making each contract explicit is more reliable than debugging only after a loss function fails.

14 Review questions and exercises

1. Why does an RNN use the same recurrent matrices at every time step?
2. For input shape `[8, 12, 50]` with `batch_first=True`, identify B , L , and D .
3. What shapes should `all_states` and `h_n` have for one layer with hidden width 64?
4. Explain why integer vocabulary IDs should not be fed directly as continuous numerical features.
5. Recompute the first manual hidden state from the four displayed parameter arrays.
6. Derive the 7,424 parameters in the recurrent layer.
7. Why does `CrossEntropyLoss` require logits $[B, V]$ and targets $[B]$?
8. What information is lost when all unseen words map to one `<UNK>` ID?
9. Why does batch size one avoid padding, and why is that not a scalable solution?
10. Give an example where maximum softmax probability above 0.5 should still be rejected.
11. Compare vocabulary-token classification, complete-answer classification, and sequence generation.
12. Design a validation set that tests both paraphrases and out-of-domain rejection.

Takeaway

A reliable PyTorch RNN pipeline begins with language and label definitions, converts them into explicit tensor contracts, verifies every axis, trains on raw logits, and evaluates both correctness and uncertainty. The recurrent cell is only one part of the system; data representation and output semantics determine what the application can actually learn and return.